DEEP LEARNING WITH CHESS

Christopher Cesar Woodhouse
Florida International University
Miami Beach, Florida
christophernasa@gmail.com

***Abstract*—This project attempts to apply deep learning to chess algorithms to produce novel and interesting engines.**

Keywords— chess, deep learning, machine learning, ai

I. RESEARCH PROBLEM

Artificial Intelligence is a large, multidisciplinary field. It is perhaps too large, leading to much confusion over its technology and applications. In this project, I will attempt to overcome my own limited knowledge by creating my own AI models.

II. MOTIVATION

While conventional algorithms are more than superior to any human chess player, there are still many reasons one might attempt to apply modern machine learning technology to chess.

For one, games such as chess provide an important testing ground for modern AI systems. This is seen in well-publicized projects such as Google's AlphaGo.

We might also be motivated to potentially produce a smaller, more compact chess engine that still plays at top levels.

Most top level chess engines also play in ways that are counterintuitive for human players. Using machine learning it may be possible to create chess engines that play in a more human manner.

It may also be possible to create chess engines that play similarly to specific chess grandmasters. Or engines that play in completely unusual ways.

III. LITERATURE REVIEW

Tom Murphy, (AKA Tom7) in his video "30 weird chess algorithms: Elo world", attempts to use various AI technologies to create novel and interesting chess engines, with the goal of creating ones that play "bad chess" well. This included a "blindfolded" algorithm that uses machine learning to predict the locations of pieces from a 8-by-8 bit binary grid.

In "Comparing traditional and deep learning approaches in developing chess AI engines." (2023), the authors compare traditional chess models like Alpha-beta pruning, quiescence search, and move ordering techniques, with modern machine learning approaches.

IV. DATASET INFORMATION

There are several datasets we can use in our ai training.

My primary dataset is this one from kaggle datasets. It is a collection of over 20,000 chess games from Lichess.org.

https://www.kaggle.com/datasets/datasnaek/chess

V. FEATURE PROCESSING AND FEATURE ENGINEERING

We transform the board state into a 8x8x14 vector. The states of each tile is one-hot encoded.

Our model is a Convolutional Neural Network

In the first version, chess moves are represented by a 64 by 64 by 5 bit vector. This is the output of the CNN model.

In the second version, moves are represented by a 64 x 64 matrix.

In the third version, I attempted to solve the model's repetitive behavior by shuffling the training data, and heavily penalizing repetitive moves in the final move prediction. This did not seem to be effective.

Next, I would process much more of the pgn data to quadruple the input dataset. This would improve model metrics but do little to solve the model's major issues.

This would be when I would discover a strange behavior. When playing against Stockfish, the model would play extremely poorly, constantly repeating the same moves. However when playing against a real person, the model would play relatively normally, at least for a while, before it would get stuck in another loop.

While I currently can't explain the difference in behavior between stockfish and human players, its repetitive behavior is likely a result of bias in the dataset. Opening moves are likely overrepresented in the dataset, and the dataset likely is missing a lot of crucial positional data since the game setup is identical in every case.

I believe I may have a solution to this problem. I will generate random boards and then have stockfish play a move. Additionally I will take boards from the dataset, randomly modify them, and then do the same. This should create a more balanced dataset that should allow the model to better grasp the positioning and movement of pieces.

VI. Deep Learning Model Building

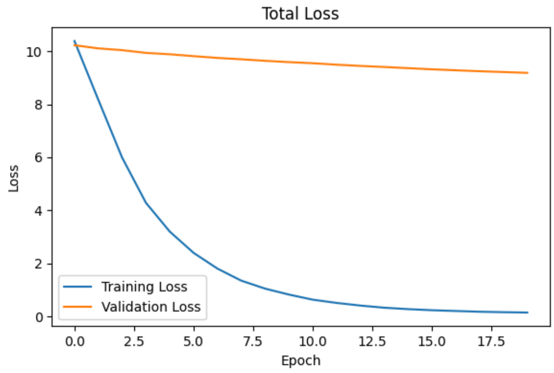
I used tensorflow to construct and train my models.

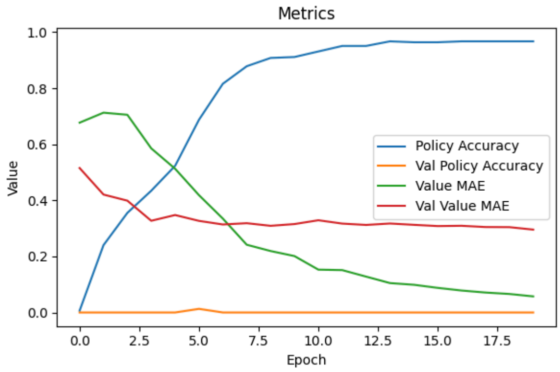
My first version uses a 64 x 64 x 5 dimensional vector to represent moves.

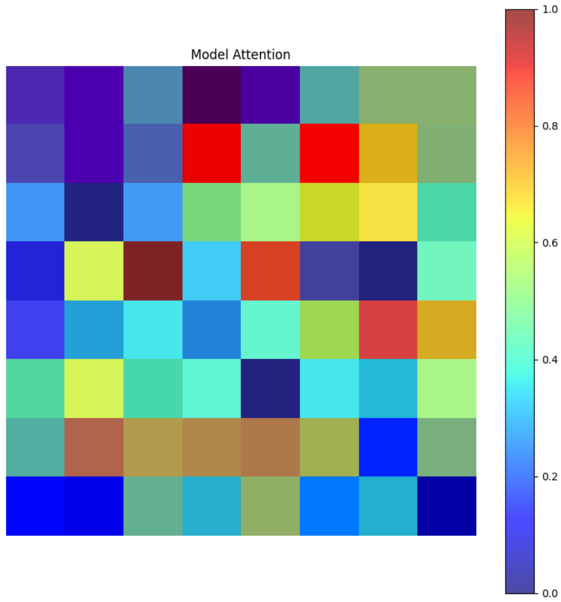
The second reduced this to 64 x 64 to hopefully improve performance.

VII. Evaluating the Result

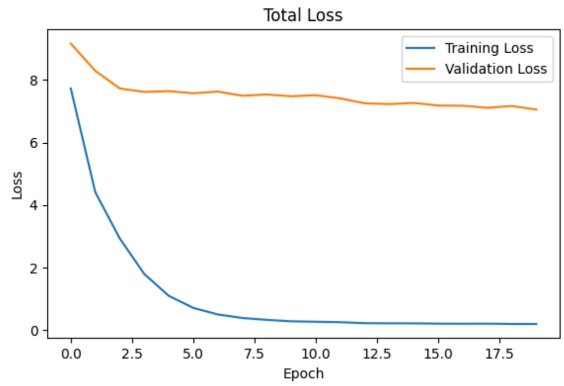
To evaluate the results of our experiment, I used standard Machine Learning measurements, alongside some specific chess measurements.

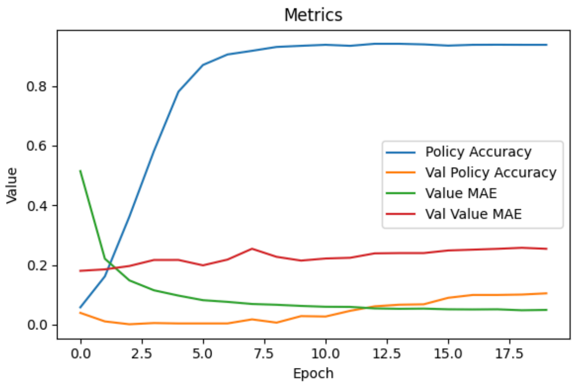


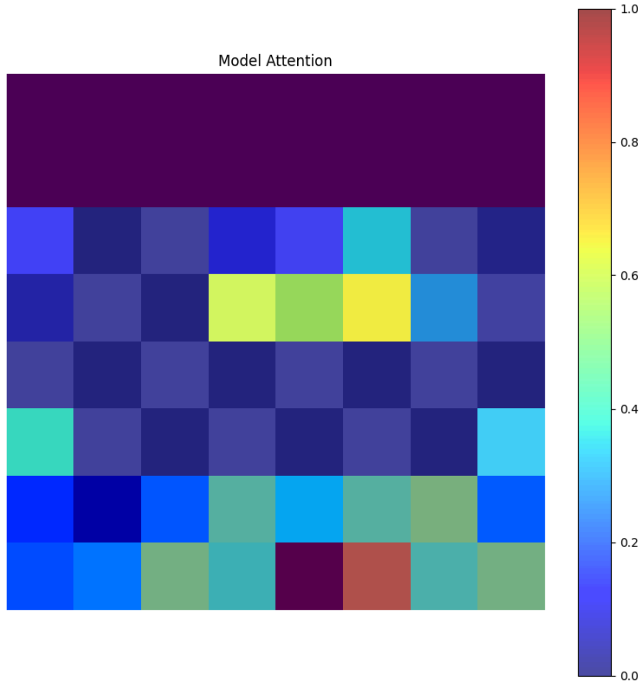


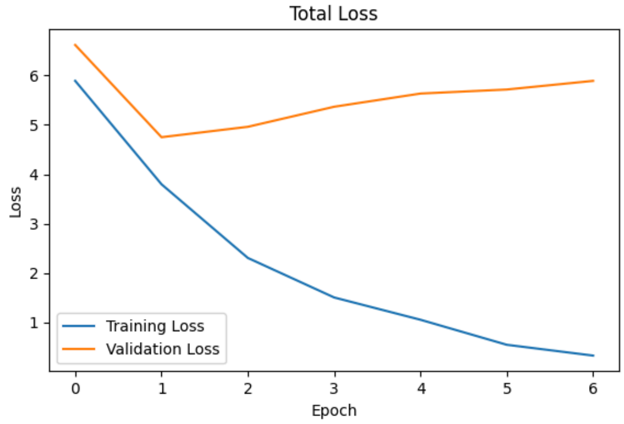


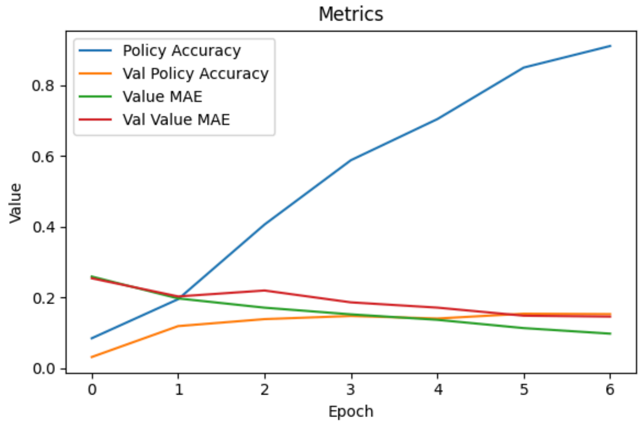
Version 1
Above is the results of the first test training I ran on the training data. Since the testing data only contained around 10 items, it seems to have just memorized the data, and does relatively poorly on new data.
This model seems to pay much more attention to other parts of the board than its successors.





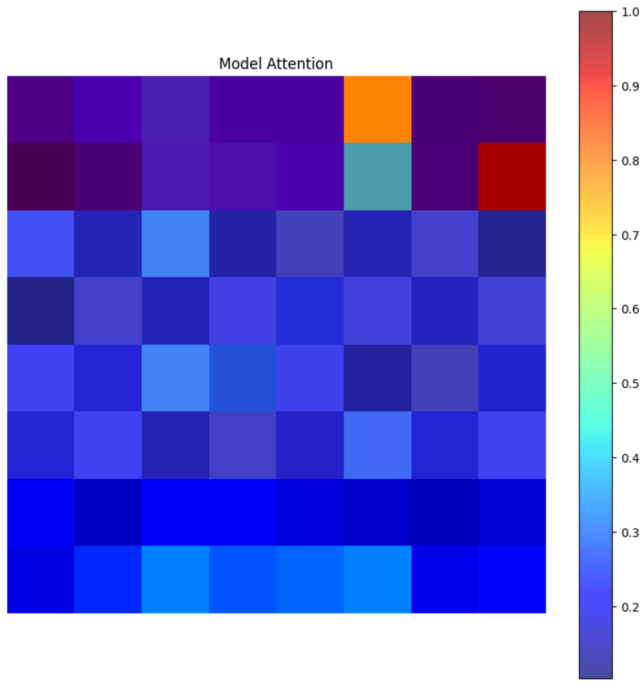






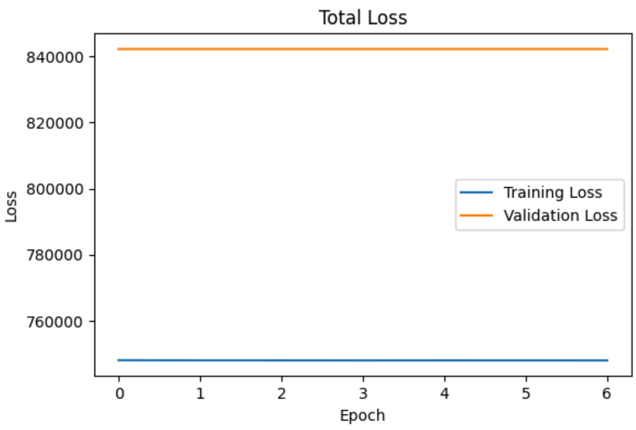
Version 2
The second version does much better on the data, however it still performs quite poorly in competition, with 0 of 5 games won against Stockfish. I found that this model tends to get stuck in a loop very quickly- moving the knight forward and then moving its rook back and forth until Stockfish puts it out of its misery. Repetitive behavior is a common problem in machine learning, and could be a result of many different problems with the model.

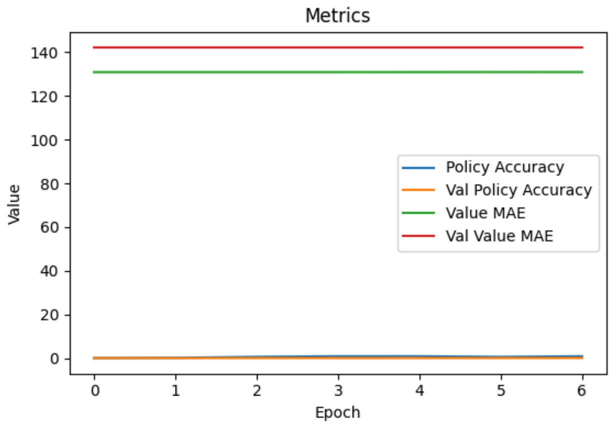
Model attention: this model seems to pay attention to a lot more of the board, including the center, which is presumably a good thing since the center is important to control in chess. It seems that the model may in fact be getting worse over time as I add more data, but i'm not sure.



Version 3
The validation loss is much lower than the prior model, however it performs very poorly against traditional models, and still suffers from severe repetitive behavior.

Model attention: the model does not seem to pay much attention to the board or its own pieces, instead focusing on their opponent.





Version 5.1
This version is trained using randomly generated boards alone. The results are not exactly promising, but they were meant to supplement the dataset, not replace it.

VIII. Conclusion

Artificial intelligence is a complicated and broad subject. Unfortunately I was unable to construct a model capable of challenging stockfish even on its lowest settings, however this has been a valuable learning opportunity.

Personally, I was surprised how much of this project was dedicated to massaging the data to improve model performance. This shows the importance of data in machine learning.

IX. Future Work

There are many avenues that could be explored. There are likely many ways to streamline the data generation and processing code I assembled, such as implementing multithreading.

Further, when I changed the model output to 64x64, this removed the ability of the model to handle promotions. This did not come up as the model tends to fall apart long before this became relevant, but making a high level model will require some dealing with promotions.

X. References

[1] Murphy, Tom. "30 Weird Chess Algorithms: Elo World." 15 July 2019, https://www.youtube.com/watch?v=DpXy041BIlA

[2] Petrick, Thomas, et al. "Building a Chess AI with Deep Learning." (2020).

[3] Abdelghani, Bassam A., Jamal Dari, and Shadi Banitaan. "Comparing traditional and deep learning approaches in developing chess AI engines." *2023 3rd International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICECCME)*. IEEE, 2023.